Estructura de Carpetas Express

- Estructura de Carpetas Express
  - Estructura mínima
  - Estructura mediana
  - Estructura amplia
  - Un solo archivo `index.js` o `app.js`
  - Ambos archivos `index.js` + `app.js`
  - Carpeta `/routes`
  - Carpeta `/Controllers`
  - Carpeta `/Models`
  - Carpeta `/Middleware`
  - Un folder por recurso

Estructura mínima

No existe una estructura correcta o default para las carpetas en una aplicación backend Node y muchos frameworks como Express no imponen ninguna pero para una aplicación minima y monolítica podríamos tener:

```
/proyecto/
        |__/node_modules/          Carpeta generada por npm conteniendo las dependencias
        ├── app.js                 Archivo principal (de entrada) index.js o app.js
        ├── .env                   Archivo con las variables de entorno
        ├── .gitignore             Archivo de git con los archivos que no deben estar en el repositorio
        ├── package.json           Archivo con información del proyecto y dependencias
        ├── package.lock.json      Archivo generado por npm para manejar dependencias
        └── README.md              Archivo Markdown con la descripción del proyecto
```

Estructura mediana

Si la aplicación crece y se van agregando mas recursos no conviene tener todo en un solo archivo
La práctica mas común es crear un router por recurso que maneje sus peticiones y cada router
invocará a su vez a un controlador que se encargará de la logia de negocio.

```
/proyecto/t
       |__/node_modules/          Carpeta generada por npm conteniendo las dependencias
       ├── app.js                Archivo principal (de entrada) index.js o app.js
       └──/routes/               Carpeta de Rutas
       |    ├── usuarios.js
       |    ├── productos.js
       |    └── pedidos.js
       └──/controllers/
            ├── usuarios.js
            ├── productos.js
            └── pedidos.js
       ├── .env                     Archivo con las variables de entorno
       ├── .gitignore            Archivo de git con los archivos que no deben estar en el repositorio
       ├── package.json          Archivo con información del proyecto y dependencias
       ├── package.lock.json     Archivo generado por npm para manejar dependencias
       └── README.md             Archivo Markdown con la descripción del proyecto
```

Estructura amplia

Conforme el servidor vaya creciendo e incrementando su funcionalidad se vuelve necesario crear mas carpetas. De nuevo, esto n oes una plantilla de facto que deba seguirse, son solo sugerencias que es común ver en muchos proyectos en la actualidad:

```
/proyecto/                          Carpeta raíz de la app
        │   app.js                  Punto de entrada de la app
        ├── /routes/                Asocia cada ruta(recurso) con su controlador
        │       ├── rec1Routes.js
        │       └── rec2Routes.js
        ├── /controllers/           Lógica de manejo de solicitudes y generar respuestas
        │       ├── rec1Controller.js
        │       └── rec2Controller.js
        ├── /models/                Interacción con la base de datos
        │       ├── rec1.js
        │       └── rec2.js
        ├── /middleware/            Funciones middleware para procesar las solicitudes
        │       ├── autentica.js    Funciones middleware de autenticación
        │       └── valida.js       Funciones middleware de Validación
        ├── /public/                Archivos estáticos que se envían al cliente
        │       ├── css/
        │       ├── js/
        │       └── images/
        ├── /services/              Servicios externos que consuma la app
        ├── /utils/                 Utilerías y funciones auxiliares
        ├── /views/                 Plantillas (templates) que "preparan" el Html para el cliente
        │       ├── rec1.pug        Cuando se usa un motor de plantillas como PUG o EJS
        │       └── rec2.pug
        └── /tests/                 Carpeta conteniendo scripts de test
                ├── unit/           Pruebas Unitarias
                ├── integration/    Pruebas de Integración
                └── e2e/            Pruebas End to End
```

Un solo archivo `index.js` o `app.js`

Archivo de entrada de la app: La inicia, configura el middleware, define rutas e inicia el servidor.
Y en el archivo **`package.json`** indicar en la propiedad **`main`** el punto de entrada de la aplicación.
y en la propiedad **`scripts:start`** el comando de arranque.

```
const express = require('express');
const routes  = require('./routes');

const app  = express();
const PORT = 3000;

app.use(express.json());                                  // Funciones Middleware
app.use(express.text());
app.use('/api', routes);                                  // Configuración de Rutas

app.listen(PORT, () => {                                 // Inicia Servidor
  console.log(`Server Express en puerto ${PORT}`);
});
```

```
{ "name": "servidor",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {  "start": "node index.js",
                "test": "echo \"Error: no test specified\" && exit 1" },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs",
  "dependencies": { "express": "^5.2.1"}
}
```

Ambos archivos `index.js` + `app.js`

El archivo **`index.js`** realizará configuraciones e importará el **`app.js`**
El archivo **`app.js`** solo inicia el servidor Express.

`index.js`

```
const app = require('./app');

app();
```

`app.js`

```
const express = require('express');
const app = express();
const port = 3000;

app.get('/', (req, res) => {
  res.send('Hola mundo');
});

app.listen(port, () => {
  console.log(`Servidor Express corriendo en puerto:${port}`);
});
```

Carpeta `/routes`

Las diferentes partes (recursos) de la aplicación serán las rutas.
Cada recurso deberá tener su enrutador.
El enrutador deberá invocar a las funciones definidas en el controlador.

```
const express = require('express');
const router  = express.Router();
const customerController = require('../controllers/customer');

router.get('/customers', customerController.getAllCustomers);

module.exports = router;
```

> Aquí se asocia cada petición (URL) a la función que se debe ejecutar
> Su única función es enrutar la solicitud (método-ruta).

Carpeta `/Controllers`

El controlador contiene la lógica para manejar las solicitudes entrantes y generar una respuesta
Cada archivo en la carpeta **`/controllers/`** corresponde a un diferente recurso o parte de la aplicación
El Controlador va a requerir el modelo, que es el responsable de la interacción con la BD.

```
const Customer = require('../models/customer');

exports.getAllCustomers = async (req, res) => {
  try {
    const customers = await Customer.find();
    res.json(customers);
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
};
```

> Aquí se colocan las funciones que realizan acciones en los recursos
> y que contienen la lógica de la aplicación.

Carpeta `/Models`

El modelo define la estructura de los datos y maneja las interacciones con la base de datos
Cada modelo corresponde a un diferente recurso.

```
const mongoose = require('mongoose');

const customerSchema = new mongoose.Schema({
  name: {
    type: String,
    required: true
  },
  email: {
    type: String,
    required: true,
    unique: true
  },
  createdAt: {
    type: Date,
    default: Date.now
  }
});

module.exports = mongoose.model('Customer', customerSchema);
```

> Aquí se colocan las funciones que interactúan con la base de datos.
> Estas funciones serán invocadas por el controlador

Carpeta `/Middleware`

En este folder se colocan funciones middleware de autenticación, validación, logs etc.

```
module.exports = (req, res, next) => {
  const token = req.header('Authorization');
  if (!token) return res.status(401).json({ message: 'Access Denied' });

  try {
    const verified = jwt.verify(token, process.env.JWT_SECRET);
    req.user = verified;
    next();
  } catch (err) {
    res.status(400).json({ message: 'Invalid Token' });
  }
};
```

> Aquí se colocan funciones auxiliares que serán usadas
> por los demás módulos.

Un folder por recurso

Aunque no es muy común de ver esta manera de organizar el código, podríamos tener una carpeta para cada recurso con todo lo necesario para manejarlo dentro de ella

```
proyecto/
    ├── index.js
    ├── productos/
    │   ├── productos.routes.js
    │   ├── productos.controller.js
    │   └── productos.model.js
    ├── proveedores/
    │   ├── proveedores.routes.js
    │   ├── proveedores.controller.js
    │   └── proveedores.model.js
    ├── middlewares/
    ├── .env
    ├── package.json
    └── package-lock.json
    └── README.MD
```